CI/CD PIPELINE

Automated Health Checks and Rollback

Using GitHub Actions, Docker, and an Azure Linux Virtual Machine

Project focus
Automated validation, container deployment, health checking, and recovery when a deployment fails.

Repository: Tharunv4444/cicd-rollback-project
Application: Node.js + Express, served on port 3000
Deployment target: Ubuntu 24.04 Azure VM

Prepared as a practical project reference and interview guide

1. Project Overview

This project demonstrates a CI/CD workflow that validates a Node.js application, builds and tests a Docker image, deploys the container to an Azure Linux virtual machine, and checks whether the deployed application is healthy. A deployment script is designed to attempt recovery by restoring the previous container image when a health check fails.

The main goal is to reduce manual deployment work and make deployment failures easier to detect and recover from. The workflow also includes failure-simulation and rollback-integration test jobs so the recovery logic can be exercised without treating a failed candidate deployment as a successful release.

2. Tools and Technologies

Technology	Purpose
GitHub Actions	Runs the automated workflow when code is pushed or the workflow is triggered.
Git and GitHub	Stores source code and version history in the project repository.
Node.js and Express	Runs the web application and provides the root and health endpoints.
npm	Installs the Node.js dependencies and supports reproducible installation with the lock file.
Docker	Packages the app into a container image and runs candidate/live containers.
Azure Linux VM	Hosts the deployed container on Ubuntu 24.04.
SSH	Allows the GitHub Actions workflow to transfer deployment files and invoke commands on the VM.
curl	Checks HTTP responses and the health endpoint.

3. Architecture and Request Flow

Developer Pushes code to GitHub
↓
GitHub Actions Install dependencies → syntax check → build image → test → deploy
↓ SSH
Azure Ubuntu VM Deployment script runs candidate container and checks /health
↓
Outcome Healthy candidate: continue deployment Unhealthy candidate/live check: attempt rollback

4. Application Endpoints

Endpoint	Expected behavior
GET /	Returns the text: `CI/CD Rollback Project is Running`.
GET /health	Returns JSON indicating the app is healthy, for example `{"status":"healthy"}`.

The health endpoint is used by deployment checks to decide whether the candidate container is responding as expected. An HTTP response alone does not prove every downstream dependency or business function is healthy; production applications may need deeper readiness checks.

5. CI/CD Pipeline Stages

- **1. Checkout source:** Each job that needs repository files checks out the repository.
- **2. Set up Node.js:** The workflow selects the Node.js runtime used by the project.
- **3. Install dependencies:** `npm ci` installs dependencies from the lock file.
- **4. Validate application:** A syntax check such as `node --check app.js` catches JavaScript syntax errors before deployment.
- **5. Build and test Docker image:** The workflow builds an image tagged with the commit SHA and starts a test container on the GitHub-hosted runner. It checks the health endpoint.
- **6. Deploy to Azure:** Over SSH, the workflow copies the application/deployment files to the VM and invokes the deployment script with the new image tag and live/candidate ports.
- **7. Simulate failure:** A separate workflow job exercises rollback behavior with a controlled failure scenario.
- **8. Integration test:** Another job copies the script to Azure and exercises the rollback path on isolated test ports. The repository checkout step is required in this job so the script and app files are available.

6. Deployment and Rollback Logic

The deployment script is designed to reduce downtime by checking a candidate container before relying on it as the live version. In the configured flow, the live application uses port 3000 and the candidate check uses port 3001. Test jobs use separate ports to avoid interfering with the live service.

- Build the candidate image from the application source.
- Start the candidate container on the candidate port.
- Poll the candidate /health endpoint until it responds successfully or the health-check timeout is reached.
- If the candidate is healthy, replace the named live container and verify the live endpoint.
- If deployment or the live health check fails, attempt to restore the previous image/container.
- If rollback succeeds after a failed deployment, return a non-zero exit status so GitHub Actions records the deployment as failed rather than marking the release as successful.

Important: rollback requires a usable previous image to remain available on the VM and the script to identify the current/previous image correctly. The implementation should be reviewed before production use, especially around container replacement timing, cleanup, image retention, secrets, and concurrent deployments.

7. Useful Verification Commands

Run commands on the Azure VM only when you understand their effect. The commands below are read-only checks unless noted.

```
# Check containers and port mappings
sudo docker ps
```

```
# Check the live app response locally on the VM
curl -i http://localhost:3000/

# Check application health
curl -i http://localhost:3000/health

# View recent container logs (replace name if needed)
sudo docker logs --tail 100 rollback-node-app

# Inspect the VM's Docker service
sudo systemctl status docker --no-pager
```

When checking a deployment, confirm the HTTP status and response body, review the container status, and inspect logs if the health check fails. From a machine outside the VM, network access also depends on the VM firewall/NSG rules and the address/port being tested.

8. Verified Project Status

Based on the project checks recorded on 14 October 2026:

- The Azure VM is running Ubuntu 24.04.4 and Docker is installed and running.
- The Node.js app container named `rollback-node-app` was reported running with a port mapping for 3000.
- The root endpoint returned `CI/CD Rollback Project is Running`.
- The health endpoint returned HTTP 200 and `{"status":"healthy"}`.
- A separate older container named `cicd-rollback-app` was also reported running on port 8080; it is separate from the Node.js application on port 3000.
- The GitHub Actions workflow was reported green after the integration-test job was updated to check out the repository.

These are point-in-time checks, not a guarantee that the service remains healthy or that every possible rollback failure has been tested. Re-run the commands and review the latest Actions run when validating the current state.

9. Resume-Ready Project Description

> I developed a CI/CD pipeline using GitHub Actions, Docker, and an Azure Linux VM. The pipeline validates a Node.js application, builds and tests a Docker image, and deploys it to Azure over SSH. I implemented automated health checks and a rollback script that attempts to restore the previous container image if the deployment fails its health check. I also added failure-simulation and integration-test jobs to verify the recovery workflow.

10. Interview Explanation

“My project is a CI/CD pipeline with automated health checks and rollback. When code is pushed to GitHub, GitHub Actions installs dependencies, checks the application, builds a Docker image, and tests the container. If the checks pass, the workflow connects to my Azure Linux VM using SSH and runs the deployment script. The script starts a candidate container, checks its health endpoint, and then proceeds with deployment. If the health check fails, it attempts to restore the previous image. I also created failure-simulation and integration-test jobs to test the rollback path.”

If asked about a key engineering consideration, explain that the health check must test the deployed candidate and that rollback depends on retaining a known-good image. A useful next step is to make deployment operations safer under concurrent runs and to add monitoring/alerts.

11. Limitations and Future Improvements

Area	Possible improvement
Image storage	Push versioned images to Azure Container Registry (ACR) rather than relying only on images stored locally on the VM.
Deployment safety	Add workflow concurrency controls, explicit timeouts, careful cleanup, and a tested strategy for retaining the last known-good image.
Health checks	Use readiness checks that cover important dependencies and meaningful application behavior, not only process availability.
Security	Store SSH keys and credentials in GitHub Secrets, use least-privilege access, rotate credentials, and avoid logging secret values.
Observability	Add centralized logs, metrics, alerts, and deployment notifications.
Release control	Add a staging environment, approval gates, and a documented recovery procedure before production releases.
Test quality	Test rollback with distinct known-good and deliberately unhealthy candidate images, and verify the old version is restored on the live endpoint.

12. Key Terms

Term	Simple meaning
CI/CD	Continuous Integration / Continuous Delivery or Deployment: automate validation and release steps.
Container image	A packaged application and its runtime dependencies used to create containers.
Health check	A request or test used to check whether the application is responding as expected.
Rollback	Return to a previous known-good version after a release problem.
SSH	An encrypted protocol for remotely connecting to a server and running commands.
GitHub Actions	GitHub’s workflow automation service.
Port mapping	Connects a port on the host to a port inside a container.

End of documentation